

Tutorial Sheet 1

Processes, threads and scheduling

AUTHOR

Operating Systems Teaching and Research Unit

PUBLISHED

08.05.2024

Introduction

Question 1

Consider the following piece of C code:

```
int main() {  
    fork();  
    fork();  
    exit();  
}
```

How many processes are created during the execution of this program?

Question 2

In a non-preemptive batch system, there are four jobs waiting to be executed with expected run times: A(9), B(6), C(3), D(5). Which scheduling algorithm should be used to minimize the average response time (latency)? What would be the optimal order for running these jobs?

Question 3

Suppose we are operating a preemptive system using the algorithm identified in the previous question. At time $t=10$, two new processes arrive, E(2) and F(7). What is the current state of the processes? Which scheduling algorithm should be employed now to minimize the average response time? What will be the sequence of execution starting from $t=10$ to achieve this goal?

Question 4

Why, in addition to the states *ready* and *running*, is there another state called *blocked*? What is achieved by this state?

Using the `top` program in the Linux shell, you can view the processes currently managed by the kernel. It provides a real-time view, including current information about process state, CPU usage, and memory usage. On a typical Linux system, you'll notice that a large number of processes are in the *sleeping* state (corresponds to *blocked* based on our definition). This does not represent the *ready* state - which is combined with the *running* state in `top`. What do you suspect: why are so many processes in the blocked state?

Tasks

Task 2.1: Processes and Process Creation

Write a simple C program following these specifications:

- Create and initialize a global variable to 0
- Inside a loop, each iteration will:
 - Increment the global variable
 - Call `fork()`
 - The child should print "Child" followed by its *pid* and the value of the global variable
 - The parent should print "Parent" followed by its *pid* and the value of the global variable
 - When the global variable reaches 5, exit the loop and exit the program
- Ensure that each parent and child processes continue to execute the loop until the variable reaches 5

Question: Observe the output, how many processes are created by the end of execution? What happens if the loop doesn't stop so early?

Task 2.2: Thread global variable increment

Modify the program from Task 2.1 to use threads instead of `fork()`. Use the pthread API (`pthread_create`, `pthread_exit`, `pthread_join`).

Question: How many threads are created in total? Why is it different from the `fork` version?

Task 2.3 Exec Syscall

Write a C program that performs a `fork()`. The child process should execute the `ls /` command using `exec1()`, while the parent waits for the child to finish and then prints a completion message.

Here is the example how to use the `exec1` syscall:

```
int exec1(const char *pathname, const char *arg, ...
          /*, (char *) NULL */);
```

```
// note that:
// - the argument after the executable should contain the
// - the last argument should be NULL
int ret = exec1("/bin/ls", "ls", "/", NULL);
```

Write a second C program that has the same behavior, except that it will create a new thread instead of a fork. The new thread will be calling the `ls /` command with `exec1()`. The main thread should be waiting for the child completion and print an exit message.

Questions: What differences do you observe between using `fork()` and creating a new thread (`pthread_create()`) in this context? What is the root cause of this behavior?

Task 2.4: Scheduling Algorithms Implementation

For this week's assignment, you will be implementing various scheduling algorithms seen during the lecture. It will include some interactive and batch scheduler.

Begin by defining C structs that can manipulate abstract programs. Consider the necessary fields these structs will require, as well as additional structures needed by an operating system to facilitate scheduling (events, states, etc.).

Some schedulers are preemptive, what additional event would be needed?

Question:

1. Define a `struct` for representing a program (`task`). What fields should this `struct` include to manage program execution and scheduling? Provide a detailed list of essential fields and their purposes.
2. Describe subsequent data structures that an operating system would use to handle scheduling. How would this structure maintain and manage the list of ready processes efficiently? Discuss possible implementations.

Bonus Task: Vigenere and Monte Carlo parallelization

Re-implement the Monte Carlo pi calculation program that was implemented during the C Bootcamp and parallelize the computation using 4 threads. This involves dividing the workload among threads to perform concurrent calculations, leading to potentially faster execution compared to a single-threaded approach. You can try to parallelize Vigenere as well.